16. Design a Rate Limiter

Key Terms:

Term	Full Name	What it does
Token Bucket	Rate limiting algorithm	Allows bursts up to bucket size, then rate-limits
Leaky Bucket	Rate limiting algorithm	Processes requests at constant rate, smooths bursts
Sliding Window	Rate limiting algorithm	Counts requests in rolling time window
429	HTTP Status Code	"Too Many Requests" - rate limit exceeded
Lua Script	Redis scripting	Atomic operations in Redis (check + update)
Fail Open	Failure mode	Allow traffic if rate limiter fails (availability > limiting)

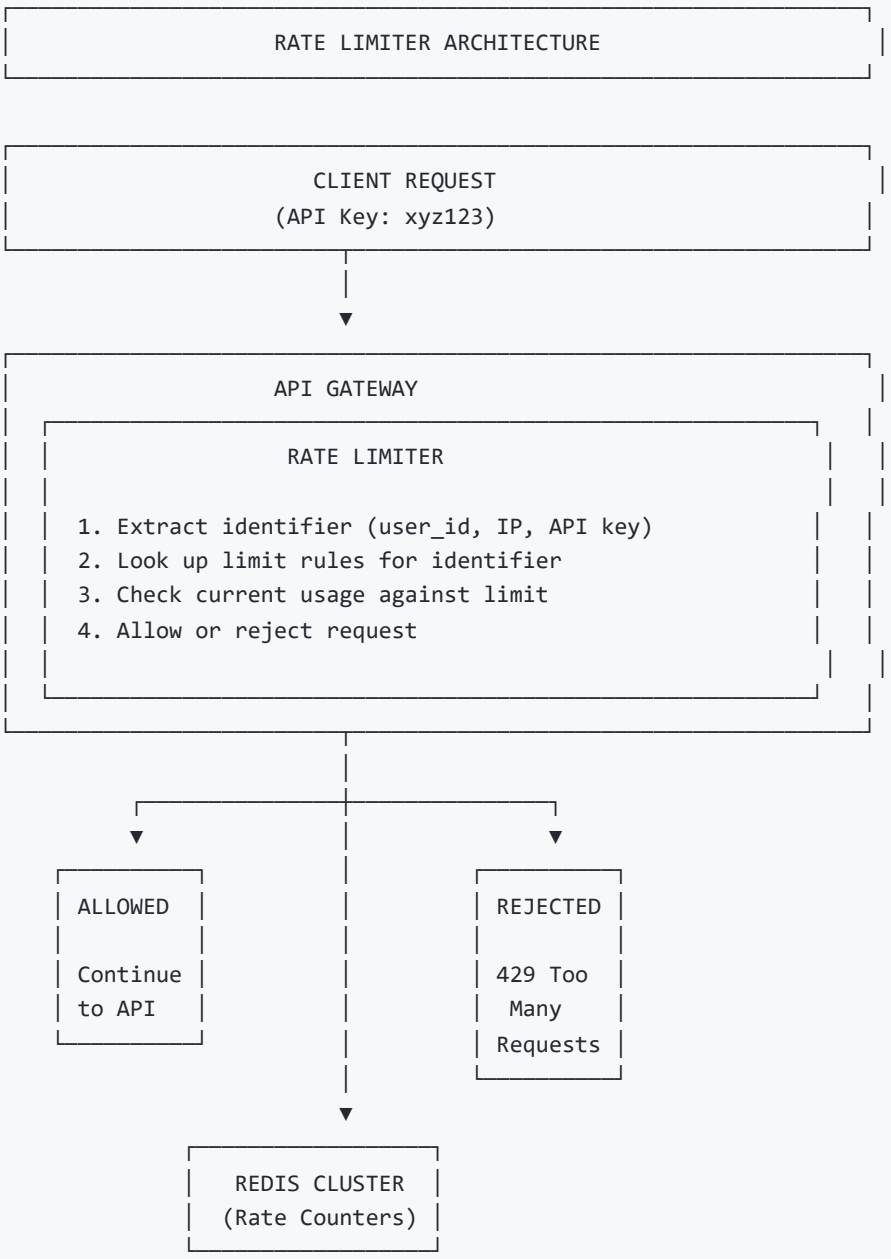
Problem Statement

Design a rate limiting system that protects APIs from abuse while allowing legitimate traffic, supporting various limiting strategies.

Clarifying Questions to Ask

1. **Limit type**: Per user, per IP, per API key, or global?
2. **Rate**: What limits? (100 req/min per user?)
3. **Granularity**: Per endpoint or global for service?
4. **Response**: Block, queue, or degrade service?
5. **Distribution**: Single server or distributed system?
6. **Accuracy**: Strict enforcement or approximate OK?
7. **Burst**: Allow short bursts above limit?

High-Level Design



API Design

Rate Limiter Configuration APIs

Method	Endpoint	Description
`POST`	`/api/v1/rules`	Create a rate limiting rule
`GET`	`/api/v1/rules`	List all rate limiting rules
`PUT`	`/api/v1/rules/{ruleId}`	Update a rule
`DELETE`	`/api/v1/rules/{ruleId}`	Delete a rule

Rate Check API (Internal)

Method	Endpoint	Description
`POST`	`/api/v1/check`	Check if request is allowed
`GET`	`/api/v1/usage?clientId={id}`	Get current usage for a client

Rate Limit Rule Example

```
POST /api/v1/rules
{
  "name": "api-standard-tier",
  "key": "client_id",
  "limits": [
    { "window": "1s", "max_requests": 10 },
    { "window": "1m", "max_requests": 200 },
    { "window": "1h", "max_requests": 5000 }
  ],
  "algorithm": "sliding_window",
  "action": "reject"  // or "throttle"
}
```

Response Headers (on every API response)

```
HTTP/1.1 200 OK                    // or 429 Too Many Requests
X-RateLimit-Limit: 200
X-RateLimit-Remaining: 150
X-RateLimit-Reset: 1672531260
Retry-After: 30                    // only on 429
```

Rate Limiting Algorithms

```
1. TOKEN BUCKET (Recommended for most cases)
┌──────────────────────────────────────────────────────────────┐
│  Bucket holds tokens, refills at constant rate               │
│  Each request consumes 1 token                               │
│  If bucket empty → reject                                    │
│                                                               │
│  Parameters:                                                  │
│    - Bucket size: 10 (max burst)                             │
│    - Refill rate: 1 token/second                             │
│                                                               │
│  Time    Tokens    Request     Result                        │
│  0.0s    10        req1        Allow (9 tokens left)         │
│  0.1s    9         req2-10     Allow (0 tokens left)         │
```

```
│  0.2s    0         req11       REJECT                          │
│  1.0s    1         req12       Allow (0 tokens left)           │
└──────────────────────────────────────────────────────────────────┘

2. SLIDING WINDOW LOG
┌──────────────────────────────────────────────────────────────────┐
│  Store timestamp of each request                                 │
│  Count requests in last N seconds                                │
│                                                                  │
│  Pros: Precise                                                   │
│  Cons: Memory-heavy (stores all timestamps)                      │
│                                                                  │
│  Redis: ZADD requests:<user> <timestamp> <request_id>            │
│         ZCOUNT requests:<user> <now - window> <now>              │
└──────────────────────────────────────────────────────────────────┘

3. SLIDING WINDOW COUNTER (Best balance)
┌──────────────────────────────────────────────────────────────────┐
│  Combine fixed window counts with weighted average               │
│                                                                  │
│  Current minute: 40 requests                                     │
│  Previous minute: 60 requests                                    │
│  Time into current minute: 30 seconds (50%)                      │
│                                                                  │
│  Weighted count = 40 + (60 * 0.5) = 70 requests                  │
│  Limit: 100/minute → Allow                                       │
└──────────────────────────────────────────────────────────────────┘

4. LEAKY BUCKET
┌──────────────────────────────────────────────────────────────────┐
│  Requests enter bucket, processed at constant rate               │
│  Smooths out bursts                                              │
│                                                                  │
│  Pros: Consistent output rate                                    │
│  Cons: Doesn't allow bursts                                      │
└──────────────────────────────────────────────────────────────────┘
```

Redis Implementation (Token Bucket)

```lua
-- Lua script for atomic token bucket (runs in Redis)
local key = KEYS[1]
local capacity = tonumber(ARGV[1])
local refill_rate = tonumber(ARGV[2])
local now = tonumber(ARGV[3])
local requested = tonumber(ARGV[4])

local bucket = redis.call('HMGET', key, 'tokens', 'last_refill')
local tokens = tonumber(bucket[1]) or capacity
local last_refill = tonumber(bucket[2]) or now

-- Refill tokens based on time passed
local elapsed = now - last_refill
```

```
    local refill = elapsed * refill_rate
    tokens = math.min(capacity, tokens + refill)

    -- Check if request can be allowed
    if tokens >= requested then
        tokens = tokens - requested
        redis.call('HMSET', key, 'tokens', tokens, 'last_refill', now)
        redis.call('EXPIRE', key, 3600)
        return 1  -- Allowed
    else
        redis.call('HMSET', key, 'tokens', tokens, 'last_refill', now)
        return 0  -- Rejected
    end
```

Response Headers

```
HTTP/1.1 429 Too Many Requests
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1705312400
Retry-After: 30

{
  "error": "rate_limit_exceeded",
  "message": "Too many requests. Please retry after 30 seconds.",
  "retry_after": 30
}
```

Scalability Considerations

- **Distributed**: Use Redis Cluster for shared state
- **Local + global**: Local rate limit first, then global check
- **Race conditions**: Lua scripts for atomic operations
- **Failover**: Allow traffic if rate limiter unavailable (fail open)
- **Multi-tier**: Different limits per tier (free, pro, enterprise)

Trade-offs

Decision	Option A	Option B	Recommendation
Algorithm	Fixed window	Sliding window	**Sliding** - smoother
Storage	Local memory	Redis	**Redis** - for distributed
Fail mode	Fail open	Fail closed	**Fail open** - availability

Precision	Exact	Approximate	**Approximate** - simpler

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
Rate Limiter	Token bucket and sliding window algorithms for traffic control	Tech Guide
Redis	Distributed counter storage with Lua scripts for atomic operations	Tech Guide

How to Explain It Well

> **Opening**: "Rate limiting protects services from abuse. I'll use token bucket for burst-friendly limiting with Redis for distributed state."

> **Key talking points**:
>
> - "Token bucket allows bursts up to bucket size, then rate-limited"
> - "Redis Lua scripts ensure atomic check-and-decrement"
> - "Sliding window counter balances precision and memory"
> - "Return standard headers: X-RateLimit-Remaining, Retry-After"
> - "Fail open if Redis unavailable - availability over rate limiting"
